Functions

Introduction **2**
What is a Function? **2**
Types of Functions **4**
1. Library Functions 4
2. User Defined Functions 5
Parts of a Function **5**
1. Function Declaration (Prototype) 5
2. Function Definition 6
3. Function Call 6
Categories of Functions **9**
Passing Arguments to a Function in C++ **13**
1. Call by Value 13
Multiple Default Arguments 20
Inline Functions **21**
Function Overloading **22**
Example 2 (Different Data Types) 24
Rules of Function Overloading **25**

Introduction

Functions are one of the most important concepts in C++. They help in writing modular, reusable, and easy-to-maintain programs.

What is a Function?

A **function** is a block of code that performs a specific task. Instead of writing the same code multiple times, we write it once inside a function and call it whenever required.

Example

```
#include <iostream>
using namespace std;

// Function to display a greeting message
```

```cpp
void greet()
{
    // Print the welcome message on the screen
    cout << "Welcome to C++ Programming";
}

int main()
{
    // Call the greet() function
    greet();

    // Indicate successful program execution
    return 0;
}
```

Output

Welcome to C++ Programming

Here,

- `greet()` is the function.
- `main()` calls the function.

Why do we use Functions?

Functions provide several advantages.

1. Code Reusability : Write the code once and use it multiple times.

2. Easy Debugging : Errors can be found and fixed easily.

3. Reduces Code Size : Avoids repetition of code.

4. Better Readability : Programs become organized and easier to understand.

5. Easy Maintenance : Updating one function updates all places where it is called.

Applications of Functions

Functions are used in almost every software application.

Examples:

- Mathematical calculations
- Banking software
- Calculator programs
- Sorting algorithms
- Searching algorithms
- Embedded systems
- Game development
- File handling
- Database operations

Example:

Instead of writing addition code repeatedly,

```
int add(int a, int b)
{
  return a+b;
}
```

can be called many times.

Types of Functions

There are mainly two types.

1. Library Functions

These are predefined functions available in C++ libraries.

Examples

sqrt()
pow()
strlen()
toupper()
tolower()

Example

```cpp
#include <iostream>
#include <cmath>   // Header file for mathematical functions like sqrt()

using namespace std;

int main()
{
    // Calculate and display the square root of 49
    cout << sqrt(49);

    // Indicate successful program execution
    return 0;
}
```

Output

7

2. User Defined Functions

Functions created by the programmer.

Example

```
void display()
{
  cout<<"Hello";
}
```

Parts of a Function

Every function has three parts.

1. Function Declaration (Prototype)

Tells the compiler

- Function name
- Return type
- Parameters

Syntax

return_type function_name(parameter_list);

Example

int sum(int, int);

Only declaration.

No body.

2. Function Definition

Contains the actual code.

Syntax

```
return_type function_name(parameter_list)
{
   // body
}
```

Example

```
int sum(int a,int b)
{
   return a+b;
}
```

3. Function Call

Invokes the function.

Example

int result;

result = sum(10,20);

Complete Example

```
#include <iostream>
```

```cpp
using namespace std;

// Function declaration (prototype)
// This tells the compiler that the add() function exists
// and takes two integer arguments and returns an integer.
int add(int, int);

int main()
{
    int ans;  // Variable to store the result of addition

    // Call the add() function with 15 and 25 as arguments
    // and store the returned result in ans.
    ans = add(15, 25);

    // Display the sum on the screen
    cout << "Sum = " << ans;

    // Indicate successful program execution
    return 0;
}

// Function definition
// This function receives two integer values and returns their sum.
int add(int x, int y)
{
    // Return the addition of x and y
    return x + y;
}
```

Output
Sum = 40

General Syntax of Function

```cpp
return_type function_name(parameters)
{
   statements;

   return value;
}
```

Example

```
float area(float r)
{
   return 3.14*r*r;
}
```

Return Type

The return type specifies what value the function returns.

Example

```
int square(int n)
{
   return n*n;
}
```

Return type = `int`

Void Function

Returns nothing.

```
void message()
{
  cout<<"Good Morning";
}
```

Function Parameters

Parameters receive values from the calling function.

Example

int add(int a,int b)

Here,

- `a`
- `b`

are parameters.

Arguments

Actual values passed during function call.

add(5,7);

5 and 7 are actual arguments.

Types of Arguments

1. Actual Arguments

Used while calling.

Example :sum(10,20);

2. Formal Arguments

Used while defining and declaring a function.

Example :int sum(int a,int b)

Categories of Functions

There are four common categories.

Type 1

No Arguments, No Return Value

A function that takes no input and performs a task without returning any value.

Meaning

- **No Arguments** → No values are passed to the function.
- **No Return Value** → The function does not return any value to the calling function.

Syntax

```
#include <iostream>
using namespace std;

// Function definition
void greet()
{
    cout << "Welcome to C++ Programming";
}

int main()
{
    // Function call
    greet();

    return 0;
}
```

Type 2

Arguments but No Return Value

A function that takes input values as arguments and performs a task without returning any value.

Meaning

- Arguments → Values are passed to the function when it is called.
- No Return Value → The function does not return any value to the calling function.

Example

```
#include <iostream>
using namespace std;

// Function definition
// a and b are called formal arguments (parameters)
void add(int a, int b)
{
    // Add a and b and display the result
```

```
    cout << a + b;
}

int main()
{
    // Function call
    // 10 and 20 are called actual arguments
    add(10, 20);

    return 0;
}
```

Type 3

No Arguments but Return Value

A function that takes no input values but performs a task and returns a value to the calling function.

Meaning

- No Arguments → No values are passed to the function.
- Return Value → The function returns a value to the calling function using return.

Example

```
#include <iostream>
using namespace std;

// Function definition
// This function takes no arguments but returns an integer value.
int square()
{
    int n;

    // Take a number as input from the user
    cin >> n;

    // Calculate the square and return the result
```

```
    return n * n;
}

int main()
{
    // Call square() function and display the returned value
    cout << square();

    return 0;
}
```

Type 4 : Arguments and Return Value

A function that takes input values as arguments, performs a task, and returns a value to the calling function.

Meaning

- Arguments → Values are passed to the function when it is called.
- Return Value → The function returns a value to the calling function using return.

Example:

```
#include <iostream>
using namespace std;

// Function definition
// This function takes two integer arguments and returns their product.
int multiply(int a, int b)
{
    // Multiply a and b and return the result
    return a * b;
}

int main()
{
    // Function call
    // 5 and 6 are actual arguments.
```

```
    // The returned value (30) is displayed using cout.
    cout << multiply(5, 6);

    return 0;
}
```

Passing Arguments to a Function in C++

When we call a function, we can pass data to it in different ways. The three important methods are:

1. Call by Value
2. Call by Reference
3. Call by Address

The main difference is whether the function works on a copy or on the original variable.

1. Call by Value

Definition: A copy of the actual argument is passed to the function, so changes made inside the function do not affect the original variable.

Example

```
#include <iostream>
using namespace std;

// Function definition
// a and b receive copies of the values
void change(int a, int b)
{
    // Changes are made to local copies
    a = 100;
    b = 200;

    cout << "Inside function: " << a << " " << b << endl;
}

int main()
{
    int x = 10;
    int y = 20;
```

```
    // x and y are actual arguments
    change(x, y);

    // Original values remain unchanged
    cout << "Inside main: " << x << " " << y;

    return 0;
}
```

Output

Inside function: 100 200

Inside main: 10 20

2. Call by Reference

Definition: A reference (alias) to the original variable is passed to the function, so changes made inside the function affect the original variable.
The & symbol is used with the parameter.

Example

```
#include <iostream>
using namespace std;

// Function definition
// a and b are reference variables
void change(int &a, int &b)
{
    // Changes are made directly to the original variables
    a = 100;
    b = 200;
}

int main()
{
    int x = 10;
    int y = 20;
```

```
    // x and y are actual arguments
    change(x, y);

    // Original values are changed
    cout << x << " " << y;

    return 0;
}
```

Output

100 200

What happens?

There is no separate copy of x and y.

x = 10 <————————> a

y = 20 <————————> b

Here: int &a

means a is another name (alias) for x.

So when we write: a = 100; it actually changes x.

Key point

Call by reference = Alias is passed → Original value can be changed.

3. Call by Address

Definition: The address of the actual argument is passed to the function using a pointer. The function uses the pointer to access and modify the original variable.

Example

```cpp
#include <iostream>
using namespace std;

// Function definition
// a and b are pointer parameters
void change(int *a, int *b)
{
    // *a accesses the value stored at the address of x
    // *b accesses the value stored at the address of y
    *a = 100;
    *b = 200;
}

int main()
{
    int x = 10;
    int y = 20;

    // Pass the addresses of x and y
    change(&x, &y);

    // Original values are changed
    cout << x << " " << y;

    return 0;
}
```

Output

100 200

What happens?

Suppose: x = 10

Address of x = 1000

When we call: change(&x, &y);

we pass the addresses:

&x ————> 1000

&y ————> 2000

The function receives them:

void change(int *a, int *b)

So:

a = 1000

b = 2000

To access the actual values, we use *:

***a = 100;**

***b = 200;**

Therefore, the original variables are modified.

Key point

Call by address = Address is passed using pointers → Original value can be changed.

Difference Between the Three

Feature	Call by Value	Call by Reference	Call by Address
What is passed?	**Copy of value**	**Reference/Alias**	**Address**
Uses pointer?	**No**	**No**	**Yes**

Uses & in parameter?	**No**	**Yes**	**No**
Uses *?	**No**	**No**	**Yes**
Can modify original?	**No**	**Yes**	**Yes**
Example parameter	`int a`	`int &a`	`int *a`
Function call	`fun(x)`	`fun(x)`	`fun(&x)`

Default Arguments

A default argument is a value automatically used if no value is supplied during the function call.

Syntax

return_type function(parameter = value);

Example

```
#include<iostream>
using namespace std;

// Function with a default argument
// If no value is passed for 'n', it will automatically use 100
void display(int n = 100)
{
    // Display the value of n
    cout << n;
}

int main()
```

```
{
    // No argument is passed to display()
    // Therefore, the default value 100 is used
    display();

    return 0;
}
```

Output

100

Passing Value

display(500);

Output

500

Multiple Default Arguments

```
#include <iostream>
using namespace std;

// Function with default arguments
// If no values are passed, a = 10 and b = 20 are used.
void show(int a = 10, int b = 20)
{
    // Display the values of a and b
    cout << a << " " << b;
}

int main()
{
    // No arguments are passed
    // Default values: a = 10, b = 20
    show();
    cout << endl;

    // Only a is passed
```

```
    // a = 50, b uses its default value 20
    show(50);
    cout << endl;

    // Both arguments are passed
    // a = 50 and b = 100
    show(50, 100);

    return 0;
}
```

Output

10 20
50 20
50 100

Rules for Default Arguments

Rule 1

Default arguments should be specified from **right to left**.

Correct : void fun(int a,int b=20,int c=30);

Incorrect : void fun(int a=10,int b,int c);

Rule 2

Default values should be given only once.

Correct : void show(int=10);

Inline Functions

Normally, whenever a function is called, the CPU jumps to that function.This process takes time. For very small functions, C++ allows the compiler to replace the function call with the actual function code. This is called **Inlining**.

Definition:
An inline function is a function for which the compiler may replace the function call with the actual code of the function at the point where it is called. This eliminates the overhead of a function call for small, frequently used functions.

Syntax

```
inline return_type function_name(parameters)
{
   statements;
}
```

Example

```
#include <iostream>
using namespace std;

// Inline function to calculate the square of a number
// The compiler may replace the function call with the function code
// to reduce function call overhead.
inline int square(int n)
{
    // Return the square of n
    return n * n;
}

int main()
{
    // Call the square() function with 5
    // and display the returned result
    cout << square(5);

    return 0;
}
```

Output

25

Advantages of Inline Function

- Faster execution

- Eliminates function call overhead
- Best for small functions

Disadvantages

- Increases program size
- Large functions should not be inline
- Compiler may ignore the `inline` request

Function Overloading

Function overloading means creating **multiple functions with the same name** but different parameter lists.

The compiler decides which function to call based on:

- Number of arguments
- Type of arguments
- Order of arguments

Example 1 (Different Number of Parameters)

```
#include <iostream>
using namespace std;

// Function with no parameters
void display()
{
    // Display a message when no argument is passed
    cout << "No parameter" << endl;
}

// Function with one parameter
// This is an example of function overloading.
void display(int a)
{
    // Display the value of the parameter
    cout << "One parameter " << a << endl;
```

```
}

// Function with two parameters
// This is another overloaded version of display().
void display(int a, int b)
{
    // Add the two parameters and display the result
    cout << "Two parameters " << a + b;
}

int main()
{
    // Calls the function with no parameters
    display();

    // Calls the function with one integer parameter
    display(10);

    // Calls the function with two integer parameters
    display(20, 30);

    return 0;
}
```

Output

No parameter
One parameter 10
Two parameters 50

Example 2 (Different Data Types)

```
#include <iostream>
using namespace std;

// Function to print an integer value
```

```
void print(int a)
{
    // Display the integer value
    cout << "Integer " << a << endl;
}

// Function to print a float value
// This is an overloaded version of print()
void print(float b)
{
    // Display the float value
    cout << "Float " << b << endl;
}

// Function to print a character value
// This is another overloaded version of print()
void print(char c)
{
    // Display the character value
    cout << "Character " << c << endl;
}

int main()
{
    // Calls print(int) because 10 is an integer
    print(10);

    // Calls print(float) because 5.5f is a float
    print(5.5f);

    // Calls print(char) because 'A' is a character
    print('A');

    return 0;
}
```

Output

Integer 10
Float 5.5
Character A

Rules of Function Overloading

Valid

- fun(int)
- fun(float)
- fun(int,int)

Invalid (only return type differs)

- int fun(int);
- float fun(int); // Error

Reason: Return type alone cannot distinguish overloaded functions.

Problem Solving on Functions

Program 1: Find Square

```
#include <iostream>
using namespace std;

// Function to calculate the square of a number
// Takes an integer argument and returns an integer value
int square(int n)
{
    // Calculate and return the square of n
    return n * n;
}

int main()
{
    // Call square() with 6 as the argument
    // Display the returned result
    cout << square(6);

    return 0;
}
```

Output

36

Program 2: Maximum of Two Numbers

```cpp
#include <iostream>
using namespace std;

// Function to find and return the maximum of two numbers
int maximum(int a, int b)
{
    // Check if a is greater than b
    if (a > b)
        // Return a if it is greater
        return a;
    else
        // Otherwise, return b
        return b;
}

int main()
{
    // Call maximum() with 25 and 10
    // Display the returned maximum value
    cout << maximum(25, 10);

    return 0;
}
```

Output

25

Program 3: Factorial Using Function

```cpp
#include <iostream>
using namespace std;

// Function to calculate the factorial of a number
int factorial(int n)
{
    int fact = 1;  // Initialize factorial result to 1

    // Loop from 1 to n
    for (int i = 1; i <= n; i++)
```

```
    {
        // Multiply fact by the current value of i
        fact *= i;
    }

    // Return the calculated factorial
    return fact;
}

int main()
{
    // Call factorial() with 5 and display the returned result
    cout << factorial(5);

    return 0;
}
```

Output

120

Program 4: Check Even or Odd

```
#include <iostream>
using namespace std;

// Function to check whether a number is even
// Returns true if the number is even, otherwise returns false
bool even(int n)
{
    // Check if n is completely divisible by 2
    return (n % 2 == 0);
}

int main()
{
    // Call even() with 12 and check the returned boolean value
    if (even(12))
        // Execute if the function returns true
        cout << "Even";
    else
        // Execute if the function returns false
```

```
        cout << "Odd";

    return 0;
}
```

Output

Even

Program 5: Area of Circle

```
#include <iostream>
using namespace std;

// Function to calculate the area of a circle
// Takes radius as an argument and returns the area as a float value
float area(float r)
{
    // Calculate and return the area using the formula π × r × r
    return 3.14f * r * r;
}

int main()
{
    // Call area() with radius 7
    // Display the returned area
    cout << area(7);

    return 0;
}
```

Output

153.86

Practice Question:

Basic Function Programs

1. **WAP to display "Hello World" using a function.**
2. **WAP to create a function to add two numbers and display the result.**
3. **WAP to create a function to subtract two numbers and display the result.**
4. **WAP to create a function to multiply two numbers and display the result.**
5. **WAP to create a function to divide two numbers and display the result.**
6. **WAP to find whether a given number is even or odd using a function.**
7. **WAP to find whether a given number is positive, negative, or zero using a function.**
8. **WAP to find the largest of two numbers using a function.**
9. **WAP to find the smallest of two numbers using a function.**
10. **WAP to find the largest of three numbers using a function.**
11. **WAP to find the smallest of three numbers using a function.**
12. **WAP to find the square of a given number using a function.**
13. **WAP to find the cube of a given number using a function.**
14. **WAP to calculate the area of a circle using a function.**
15. **WAP to calculate the area of a rectangle using a function.**
16. **WAP to calculate the area of a square using a function.**
17. **WAP to calculate the perimeter of a rectangle using a function.**
18. **WAP to calculate the simple interest using a function.**
19. **WAP to calculate the factorial of a given number using a function.**
20. **WAP to check whether a given number is prime or not using a function.**

Number-Based Function Programs

21. **WAP to find the sum of digits of a given number using a function.**
22. **WAP to find the reverse of a given number using a function.**
23. **WAP to check whether a given number is a palindrome or not using a function.**
24. **WAP to count the number of digits in a given number using a function.**
25. **WAP to find the power of a number using a function.**
26. **WAP to find the sum of natural numbers from 1 to N using a function.**
27. **WAP to find the sum of even numbers from 1 to N using a function.**
28. **WAP to find the sum of odd numbers from 1 to N using a function.**
29. **WAP to check whether a given number is an Armstrong number or not using a function.**
30. **WAP to check whether a given number is a perfect number or not using a function.**

Array-Based Function Programs

31. **WAP to input and display array elements using a function.**
32. **WAP to find the sum of all array elements using a function.**
33. **WAP to find the average of array elements using a function.**
34. **WAP to find the largest element in an array using a function.**
35. **WAP to find the smallest element in an array using a function.**
36. **WAP to count the even elements in an array using a function.**

37. **WAP to count the odd elements in an array using a function.**
38. **WAP to search for a given element in an array using a function.**
39. **WAP to reverse an array using a function.**
40. **WAP to sort an array in ascending order using a function.**

Function Concept Practice

41. **WAP to swap two numbers using a function.**
42. **WAP to swap two numbers using call by reference.**
43. **WAP to swap two numbers using call by address.**
44. **WAP to increment a number by 10 using call by value.**
45. **WAP to increment a number by 10 using call by reference.**
46. **WAP to increment a number by 10 using call by address.**
47. **WAP to display a number using a function with a default argument.**
48. **WAP to calculate the area of different shapes using function overloading.**
49. **WAP to create a calculator using separate functions for addition, subtraction, multiplication, and division.**
50. **WAP to create a menu-driven program using functions to perform different arithmetic operations.**